

Evaluating Matrix Tiling Multiplication For Asymmetric Data Cost

Project Aleph Spring 2026

Regev Ginzburg, Areg Mikhtyrian

October 02, 2026

Contents

1	Introduction	1
1.1	Background: Caches, Memory, Matrix Multiplication	1
1.2	Importance Of Asymmetric Data Access Model	1
1.3	Theoretical Results	2
2	Methodology	3
2.1	<code>asymm_tiling</code> Simulator	3
2.1.1	Our Model	3
2.1.2	Architecture	4
2.1.3	Correctness	6
2.2	Simulation Parameters: Soundness And Real-Life References	8
3	Experiments And Results	10
3.1	Reproducing The Original Paper	10
3.2	<i>B</i> -Stationary Against <i>C</i> -Stationary	10
3.3	The Memory Cost α	12
3.4	Memory Cost Against Generation Cost	14
3.5	Refining The Model	15
3.6	Choosing The Tile	15
3.7	Set-Associative Caches	17
4	Discussion	18
5	Conclusion	19
	Bibliography	20
6	Appendix	21
6.1	Implementation & Build Environment	21

1 Introduction

1.1 Background: Caches, Memory, Matrix Multiplication

Caches. A processor computes much faster than main memory (DRAM) can deliver data. Small, fast caches close to the core hold recently used data, so that repeated uses of the same data are served quickly [1]. In the configurations of this report, an L_1 hit costs 4 cycles while a trip to memory costs 180. Data moves between memory and the cache in **lines** (64 bytes). A cache is divided into **sets**: the address of a line decides which set it belongs to, and each set holds a fixed number of lines, its **associativity** (or number of **ways**). A **fully associative** cache has a single set, so any line can go anywhere. When a set is full, the **replacement policy** (typically LRU, least recently used) chooses which line to evict. Misses come in three kinds: **compulsory** misses on the first use of a line, **capacity** misses when the data in use is larger than the cache, and **conflict** misses when too many lines map to the same set although the cache as a whole has room.

Tiled matrix multiplication. Computing $C = A \cdot B$ with $A \in \mathbb{R}^{M \times K}$ and $B \in \mathbb{R}^{K \times N}$ takes MNK multiply-accumulates (MACs) over only $MK + KN + MN$ elements: every element of A is used N times and every element of B is used M times. A straightforward loop nest streams through the matrices and, once they no longer fit in the cache, loads each element from memory over and over. **Tiling** (blocking) splits the computation into tiles small enough to stay in the cache, so that a loaded element is reused many times before it is evicted. When all matrices cost the same to access, square tiles are optimal, and no schedule can move fewer than on the order of $MNK/\sqrt{S}$ elements for a fast memory of S elements [2]. Tiling is applied at two levels: cache tiles of $T_M \times T_N$ output elements, and within them register tiles of $R_M \times R_N$ elements, the unit that is loaded into the registers and multiplied at once.

Dataflows. A tiled loop nest is described by which operand stays in place (is **stationary**) in the inner loops while the others stream through, a terminology from the accelerator literature [3]. In the **output-stationary** dataflow, a block of C stays in the registers and accumulates over the whole inner dimension K . In the **weight-stationary** dataflow, a block of B stays in the registers and is reused across all T_M rows of a tile of A , while A and C stream through the cache. In this report B is the generated matrix, so weight-stationary means B -stationary.

1.2 Importance Of Asymmetric Data Access Model

Multiplying data by a random matrix is a core building block of randomized numerical linear algebra (RandNLA) [4], [5], [6], [7]. A random projection, or **sketch**, compresses a large matrix into a much smaller one while approximately preserving its geometry. By the Johnson–Lindenstrauss lemma [8], distances between n points survive a random projection to only $O(\log n)$ dimensions. Sketching underlies fast algorithms for low-rank approximation and least-squares problems [4], [5], [6]. In machine learning, random features approximate kernel methods by a multiplication with a random matrix [9].

In all of these algorithms the random matrix B carries no information of its own: any matrix drawn from the right distribution works. This makes it much cheaper to provide than the data it multiplies, in two ways:

1. **Low precision:** Random matrices with entries of only ± 1 already give Johnson–Lindenstrauss guarantees [10]. An element of B can be stored in a single bit or byte, while A and C keep full precision. This is an asymmetric-precision matrix multiplication, which reduces cache traffic, as shown in theory (Section 1.3) and in our experiments (Section 3.1).
2. **No storage at all:** B can be generated on the fly from a small seed by a pseudo-random number generator (PRNG), as the computation consumes it. B then causes no memory traffic at all, only generation time.

Either way, accessing B costs much less than accessing A and C , which is the asymmetric-access-cost setting this report studies. The second option, a hardware PRNG streaming B through a FIFO, is the focus of our model and experiments.

1.3 Theoretical Results

This project builds on an analysis of $C = A \cdot B$ when one input matrix is much cheaper to bring into fast memory than the others [11], [12]. The motivating case is a random matrix B (for example a random projection, Section 1.2), whose entries can be regenerated from a seed instead of being loaded from memory. Bringing an element of B into fast memory then costs only a fraction $\rho \leq 1$ of the cost of an element of A or C . (The same analysis applies when B is merely stored at a lower precision.) The original analysis calls the cheap matrix A ; we call it B throughout.

Tiling scheme. Keep one block of C of $T_M \times T_N = S$ elements in a fast memory of S elements, and stream in the T_M rows of A and the T_N columns of B that contribute to it, one rank-1 update at a time. Each of the MN/S blocks costs $T_M K$ for A and $\rho K T_N$ for B , and every element of C is read once, so the total cost is

$$Q = MNK \left(\frac{1}{T_N} + \frac{\rho}{T_M} \right) + MN.$$

Minimizing Q for a fixed block size S gives

$$T_M = \sqrt{\rho S}, \quad T_N = \sqrt{\frac{S}{\rho}}, \quad \frac{T_N}{T_M} = \frac{1}{\rho}, \quad Q^* = \frac{2MNK\sqrt{\rho}}{\sqrt{S}} + MN.$$

The optimal tile is therefore not square: it is stretched by a factor of $1/\rho$ along the cheap matrix. Compared with a square tile ($T_M = T_N = \sqrt{S}$, cost $(1 + \rho)/\sqrt{S}$ per MAC), the saving is $1 - 2\sqrt{\rho}/(1 + \rho)$, for example 20% at $\rho = 1/4$ and 37% at $\rho = 1/8$.

Lower bound. The second reference shows that no schedule can move fewer than on the order of $MNK\sqrt{\rho}/\sqrt{S}$ words [12]. The tiling above is therefore asymptotically optimal, and a cheap matrix reduces data movement by a factor of $\sqrt{\rho}$ compared with standard matrix multiplication.

From traffic to time. This analysis counts words moved. When B is produced by a hardware PRNG, its cost is no longer memory traffic but **generation time**, which runs in parallel with loading A . This is what leads to the cycle model of our Methodology. Section 3.1 confirms the traffic results above in our simulator.

2 Methodology

2.1 `asymm_tiling` Simulator

To evaluate matrix multiplication under asymmetric data access costs, we designed and implemented `asymm_tiling`, a modular, cycle-accurate architectural simulator written in C++20. The simulator models the execution of General Matrix Multiply (GEMM) operations across configurable multi-level cache hierarchies, comparing conventional DRAM-backed dataflows against hardware-accelerated on-the-fly streaming.

2.1.1 *Our Model*

We model mixed-precision GEMM computing $C = A \cdot B$, where:

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}, \quad C \in \mathbb{R}^{M \times N}$$

In our asymmetric setup, input matrix A and accumulator/output matrix C are represented in high precision (e.g., FP32, INT32 or even INT64) we define their precision to be P_A . Matrix B on the other hand, resides in low precision (e.g., INT8) and we define its precision to be P_B . We define the precision asymmetry ratio as:

$$\rho = \frac{P_B}{P_A} \leq 1$$

We model two distinct operational paradigms for matrix element access:

1. **Conventional DRAM-backed Access:** Both operands A and B reside in simulated main memory and must be fetched through the memory hierarchy (L_1 , L_2 , L_3 ,... and DRAM). Under this regime, reducing P_B reduces the cache footprint of the matrices and their bandwidth demand by a factor of $\frac{1}{\rho}$.
2. **Hardware PRNG-FIFO Streaming:** Matrix B is generated on-the-fly in dedicated hardware from compact seeds that are stored in memory. It's generated via a Pseudo-Random Number Generator (PRNG) coupled to an asynchronous FIFO queue. Under this model, the elements of matrix B bypass the cache and DRAM subsystem entirely, incurring zero cache pollution and near-zero memory traffic.

The overall execution time T normalized per Multiply-Accumulate operation (MAC) is governed by two concurrent processes: memory latency and hardware generation rate:

$$\frac{T}{MNK} = \max \left(\alpha(T_M, T_N), g_c \cdot \frac{\lceil \frac{M}{T_M} \rceil}{M} \right) + \frac{g_c \cdot R_M R_N \cdot \lceil \frac{M}{T_M} \rceil \lceil \frac{N}{T_N} \rceil}{MNK}$$

where:

- $\alpha(T_M, T_N)$ is the **memory-bound** cost (cycles per MAC) determined by cache miss rates and DRAM transfer latencies for a given tile shape ($T_M \times T_N$).
- g_c is the **hardware PRNG generation latency** (cycles per element of B). In the weight-stationary dataflow, each generated element of B is reused across all the rows of a tile of A , and B is regenerated once per row of tiles. The $\lceil M/T_M \rceil \cdot KN$ generated elements,

spread over MNK MACs, cost $g_c \cdot \lceil M/T_M \rceil / M$ cycles per MAC. When T_M divides M this is simply g_c/T_M .

- The max operator reflects the asynchronous overlap between background FIFO generation and foreground compute/memory access: it assumes the two overlap perfectly, so whichever is slower sets the runtime. The formula assumes that T_M divides M , so that every row of tiles is the same. Section 3.5 shows how to extend it to the other tile heights.
- The last term is the FIFO **startup** cost. The FIFO restarts from a new seed at every tile, so the first register block of R_MR_N elements must be generated before any overlap can begin. That is $g_c \cdot R_MR_N$ cycles for each of the $\lceil M/T_M \rceil \lceil N/T_N \rceil$ tiles.

The same form applies to the output-stationary (C -stationary) dataflow, with a different B term. There, each block of C stays in the registers while B streams past it, so B is regenerated for every register block of R_M rows, at a cost of g_c/R_M per MAC. Since $T_M \gg R_M$, the weight-stationary dataflow amortizes generation over many more rows, which is why it is the focus of this report.

Table 1 lists the units of every quantity in the formula. Both B terms are a generation time per element multiplied by a number of generated elements per MAC:

$$\frac{\text{cycles}}{\text{element}} \cdot \frac{\text{elements}}{\text{MAC}} = \frac{\text{cycles}}{\text{MAC}}$$

which is the unit of $\frac{T}{MNK}$ and of α . All three terms are therefore comparable, which is what allows taking their maximum and adding the startup term.

Quantity	Meaning	Units
T	total runtime	cycles
MNK	scalar multiply-accumulates	MACs
$\alpha(T_M, T_N)$	runtime with free generation ($g_c = 0$), divided by MNK	cycles / MAC
g_c	time to generate one element of B	cycles / element
$\lceil M/T_M \rceil \cdot KN$	elements of B generated (once per row of tiles)	elements
$\lceil M/T_M \rceil / M$	the same, divided by MNK	elements / MAC
$R_MR_N \cdot \lceil M/T_M \rceil \lceil N/T_N \rceil$	elements generated before overlap starts (one register block per tile)	elements

Table 1: Units of the quantities in the model formula.

Section 3.5 documents step by step how we arrived at this formula from the original $\max(\alpha, \frac{g_c}{T_M})$, and how well each part matches the simulator.

2.1.2 Architecture

The simulator is structured into decoupled, modular subsystems synchronized by a centralized event clock:

- **MultiUnit (Compute & Tiling Engine):** Orchestrates the 3D loop nest (M, K, N) partitioned into tiles of $T_M \times T_N$ outputs (each tile spans the full inner dimension K) and register tiles ($R_M \times R_N$, typically 4×4). Tiles at the matrix edges, and register blocks at the tile edges, are clipped to what is left, so tile sizes need not divide the matrix dimensions.

It supports both **Output-Stationary** (C -stationary, accumulating outputs locally while streaming inputs) and **Weight-Stationary** (B -stationary, holding stationary weight tiles to maximize B reuse) dataflows. The matrices A , B and C are laid out back to back from a fixed base address, so every run is deterministic.

- **PrngFifo (PRNG-FIFO Hardware Accelerator):** Models an asynchronous generator in front of a FIFO of finite capacity (in elements). In the background, the generator produces one element of B every g_c cycles and pauses while the FIFO is full. The compute core pops one register block ($R_M R_N$ elements) at a time: if the FIFO holds fewer elements, the core stalls until the missing ones are generated, and every pop costs a fixed access latency. Each tile starts by loading its seed, which restarts the generator with an empty FIFO.
- **CacheUnit (Multi-Level Cache Controller):** Coordinates an extensible Chain-of-Responsibility hierarchy of arbitrary depth ($L_1, L_2, \dots, L_k$, DRAM). The configuration currently describes an L_1 and an optional L_2 .
 - **CacheLevel:** Manages tag/index/offset address translation, hit latency accounting, and recursive backfill on misses. Any size that splits into a power-of-two number of sets is supported, including fully associative caches.
 - **Set (replacement):** Every line carries an age stamp, and the replacement policy (LRU, FIFO, MRU or Random) decides whether a hit refreshes the stamp and which line is the victim. The ways of a set are a contiguous array, which is faster than a hash map for typical associativities. Random replacement uses a fixed per-set seed, so runs are reproducible.

The cache follows these rules:

- **Accesses:** Reads and writes both count as accesses (and as hits or misses) at every level they reach. A level is reached only if every level above it missed.
- **Latency:** Lookups are serial. An L_1 hit costs the L_1 latency, an L_2 hit the L_1 plus L_2 latencies, and a miss everywhere adds the memory latency. Reads and writes cost the same.
- **Write policy:** All levels are write-back, with a dirty bit per level. With Write-Allocate, a write is handled like a read and then marks the L_1 line dirty. With No-Write-Allocate (write-around), the write goes down level by level: the first level that hits marks its line dirty, and if every level misses it is a single memory write. Nothing is allocated.
- **Inclusion:** The hierarchy is non-inclusive non-exclusive (NINE). A miss fills every level that missed, lowest level first, and an eviction from a lower level does not invalidate the upper levels.
- **Dirty evictions:** A dirty line evicted from one level is written to the next. If that level holds the line, it is marked dirty in place. If not, it is installed there as dirty without fetching it from memory, since the whole line is being written. A dirty line evicted from the last level is a memory write.
- **Writebacks:** Writebacks are modeled as off the critical path, as with a write buffer. They cost no cycles and are not counted as accesses, but they are counted as writebacks and memory writes.
- **End of run:** The cache is not flushed, so dirty lines left at the end are not written back.
- **Clock (Cycle Synchronization):** A centralized simulation clock (Clock) that advances monotonically with MAC compute latencies, cache lookup costs, DRAM transfer penalties, and PRNG-FIFO pipeline stall cycles.
- **Registry (Statistics Collection):** A centralized metrics registry (Registry) that records and computes dynamic simulation statistics (e.g., hits, misses, local/global miss rates,

FIFO stall rates, total cycles, and average access times) via registered callbacks, decoupling statistics gathering from core simulation logic.

- **Config and MyOptions (Configuration & Parameter Management):** Implements a two-tiered parameterization system. **Config** loads baseline architectural profiles from declarative TOML files (`config.toml`), while **MyOptions** (extending `CLI::App`) parses command-line arguments to dynamically override parameters on the fly, enabling scripted parametric sweeps without recompiling the simulator.

The relationship between these subsystems is illustrated in the UML class diagram in Figure 1.

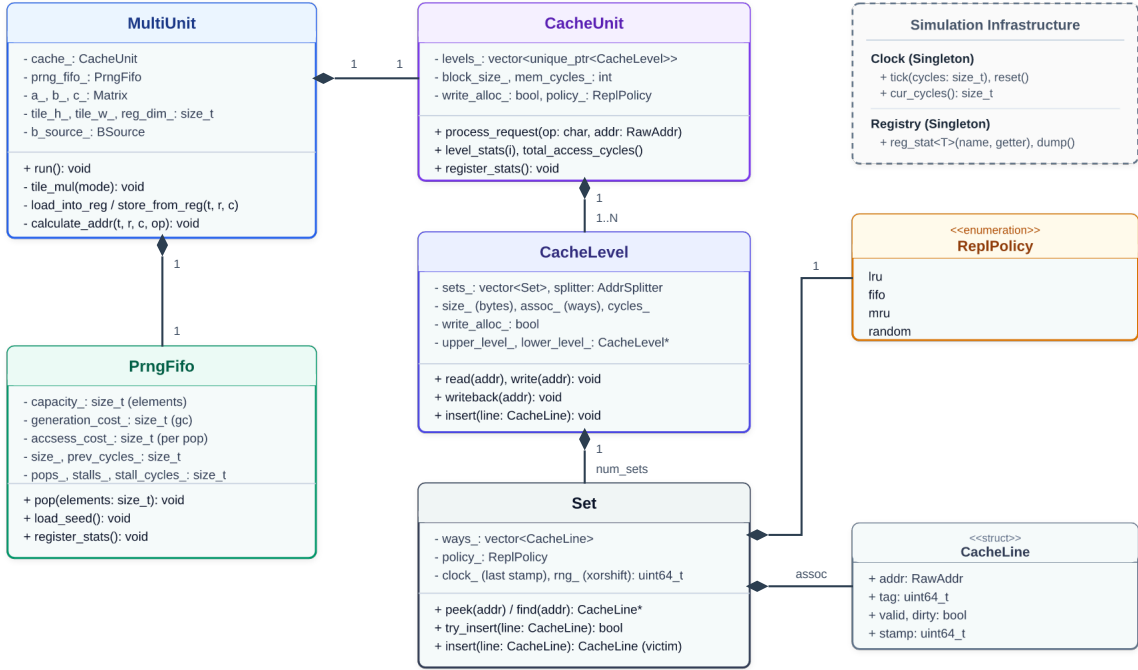


Figure 1: UML class diagram of the *asymm_tiling* simulator, illustrating the separation of the computation engine (*MultiUnit*), accelerator pipeline (*PrngFifo*), modular cache hierarchy (*CacheUnit*), and eviction policies.

To ensure high performance, maintainability, and clean separation of concerns, the simulator is implemented in standard C++20 and utilizes lightweight, header-only third-party libraries for infrastructure: `CLI11` for parsing command-line parameters, `toml++` for configuration file management, and `fmt/spdlog` for fast formatted console and log output. Detailed build instructions, compiler requirements, and library dependencies are summarized in Section 6.1.

2.1.3 Correctness

A simulator is only useful if its numbers can be trusted, so we test it at three levels: the cache on its own, the whole simulator, and the simulator against the previous version of our simulator. The test suite (144 tests, written with `pytest`) runs in about 20 seconds after every change.

A reference cache. The cache rules listed in the Architecture section are the specification. We implemented them a second time, independently, as a small Python model (about 200 lines). It is written as simply as possible: each set is an ordered dictionary keyed by the full line address, so it shares no tag/index arithmetic and no replacement code with the C++ cache. A small

driver program feeds the C++ `CacheUnit` a trace of reads and writes and prints, for every access, its cycles, the hit or miss at every level, and the number of memory accesses it caused.

Differential testing. The C++ cache and the reference model run the same traces and are compared access by access, reporting the first access where they disagree. The traces come from three sources:

- Random traces from the property-based testing library `Hypothesis`. It generates hundreds of traces per run, kept within a small address window so that sets conflict, with working sets below, around and above the cache capacity. When a trace fails, Hypothesis shrinks it to a minimal failing trace, usually a handful of accesses.
- Sequential sweeps over twice the cache capacity, in which every fourth access is a write.
- Hand-crafted traces for rare paths, such as a dirty line that must be written into an L_2 which has already evicted it.

Every comparison is repeated over direct-mapped, set-associative and fully associative caches, caches whose size is not a power of two, configurations with and without an L_2 , all four replacement policies, and both write-allocate settings.

Invariants. The reference model is itself only trustworthy if it is right, so we also test it against properties that must hold for any trace, independently of how it is implemented:

- $\text{Hits}(L_k) + \text{Misses}(L_k) = \text{Accesses}(L_k)$, and every miss of L_k is exactly one access of L_{k+1} (or of memory, below the last level).
- The cycles of every access follow the latency rules, and the total cycles equal $\sum_k \text{Accesses}(L_k) \cdot \text{latency}(L_k)$ plus the memory latency times the number of memory reads and demand writes. The driver additionally checks that the C++ cache’s own cycle statistic equals the cycles it actually charged.
- Read-only traces never create dirty lines, and no set ever holds more lines than it has ways, or a line that belongs to another set.
- No write is ever lost: after the last write to a line, the line is either still dirty somewhere or has been written to memory. Dirty data is never duplicated.
- LRU is checked against a second, completely different algorithm, Mattson’s stack-distance algorithm, which decides hits and misses without any eviction logic. Adding ways never adds misses (the inclusion property of LRU).

The whole simulator. For small matrices that fit in the cache, the expected statistics follow from the matrix layout alone. Every miss must be a compulsory miss, so the misses equal the number of cache lines covered by A , B and C . The FIFO pops match the loop structure of each dataflow, and the total cycles equal the cache cycles plus the multiply-accumulate cycles. Tiles that do not divide the matrix are tested in two ways. The output-stationary work must not depend on the tiling at all. With every matrix resident in the cache, the misses must equal exactly the lines of the matrices, which detects any access outside a matrix. Further tests cover the configuration rules (cache geometries that cannot be built are rejected with a clear error) and the units of every parameter.

The previous simulator. Finally, we compared the new simulator with the previous version of our simulator, written independently of the new code. On the experiments both can express, the L_1 traffic of the two simulators is identical in most runs (for example 70 of 80 runs of the α calibration grid) and differs by a few percent at most in the others. The paper’s results are reproduced exactly (Section 3.1).

Bugs found and regressions. During development the differential tests found two bugs in the first version of the cache, both fixed since: writes were neither counted nor charged any cycles, and a dirty line written back to L_2 was stored with the tag computed by L_1 , which caused false L_2 hits. Every later change to the simulator was checked by re-running all previous experiments (about 6000 runs) and requiring identical results, unless the change was meant to alter them.

2.2 Simulation Parameters: Soundness And Real-Life References

Unless stated otherwise, every experiment uses the configuration in Table 2, which is also the simulator’s default `config.toml`. Experiments that vary a parameter say so.

Parameter	Value	Real-life reference
Matrices $M \times K \times N$	$192 \times 256 \times 256$ (other shapes in Section 3.5)	small enough for thousands of runs, large enough that the matrices do not fit in L_1
Element size	4 bytes for A , B and C (B varies in the paper reproduction)	FP32 / INT32
Register tile $R_M \times R_N$	4×4	small register tiles of vector and matrix units
L_1 cache	16 KB, 64-byte lines, fully associative, LRU, write-back, write-allocate	CPU L_1 data caches: tens of KB, 64-byte lines, 4- to 12-way; accelerators: software-managed scratchpads
L_1 hit latency	4 cycles	a few cycles [1]
L_2 cache	none (64 KB, 14 cycles in the paper reproduction)	see below
Memory latency	180 cycles	DRAM: one to a few hundred cycles [1]
PRNG generation cost g_c	swept, 0 to 600 cycles per element	from a fast dedicated generator to a slow or shared one
FIFO capacity	16384 elements (never fills)	large enough not to limit the overlap
FIFO access	2 cycles per register block	comparable to an L_1 access
MAC cost	0 (not counted)	see below

Table 2: Default simulation parameters.

Fully associative L_1 . A device with a hardware PRNG for B is an accelerator, and accelerators usually keep their working data in a software-managed scratchpad rather than a set-associative cache. A scratchpad has no conflict misses, and a fully associative cache is the closest cache model of it. It is also the “ideal fast memory” assumed by the theory of Section 1.3, which is why the paper’s results reproduce exactly. Finally, it isolates the effects we study (tile shape and generation cost) from conflict misses, which depend on how the matrices happen to be aligned in memory. For example, with power-of-two row strides, the rows of a tile column can all fall

into the same few sets of a set-associative cache. Section 3.7 shows how the results change on 8-way and 4-way caches, which are typical of CPU L_1 caches.

No L_2 . The model distinguishes two costs for A : an L_1 hit or a trip to memory. With a single cache level, α has a direct interpretation: the cost of A 's misses to memory plus its hits in L_1 . An L_2 would add a third, intermediate cost. The paper reproduction keeps an L_2 to match the setup of the original experiments.

Latencies. What matters for the tiling decisions is mainly the ratio between a memory access and an L_1 hit, here $180/4 = 45$. That is the order-of-magnitude gap between DRAM and L_1 in current processors [1].

Multiply-accumulate cost. The MAC cost is set to 0 so that α measures only the cost of moving A and C , as in the original experiments. The dataflow performs (up to the clipped edge blocks) the same number of register multiply-accumulates for every tiling. A nonzero MAC cost would therefore add the same constant per MAC to α for every tile. It is not entirely neutral, though: a larger α moves the point where generation becomes the bottleneck.

Determinism. The matrices are placed back to back from a fixed, line-aligned base address, and random replacement (when used) has a fixed seed, so every run is reproducible.

3 Experiments And Results

3.1 Reproducing The Original Paper

3.2 B -Stationary Against C -Stationary

The two dataflows differ in how often they generate B . The weight-stationary (B -stationary) dataflow generates B once per row of tiles, so each element is reused by T_M rows of A . The output-stationary (C -stationary) dataflow keeps a register block of C while B streams past it, so B is generated again for every register block of $R_M = 4$ rows. Figure 2 compares them on the tile $T_M = 64$, $T_N = 32$, for every integer g_c from 0 to 100, with the default configuration otherwise (Table 2). The dashed lines are the model of each dataflow: $\max(\alpha_B, g_c/64)$ and $\max(\alpha_C, g_c/4)$, with α_B and α_C measured at $g_c = 0$.

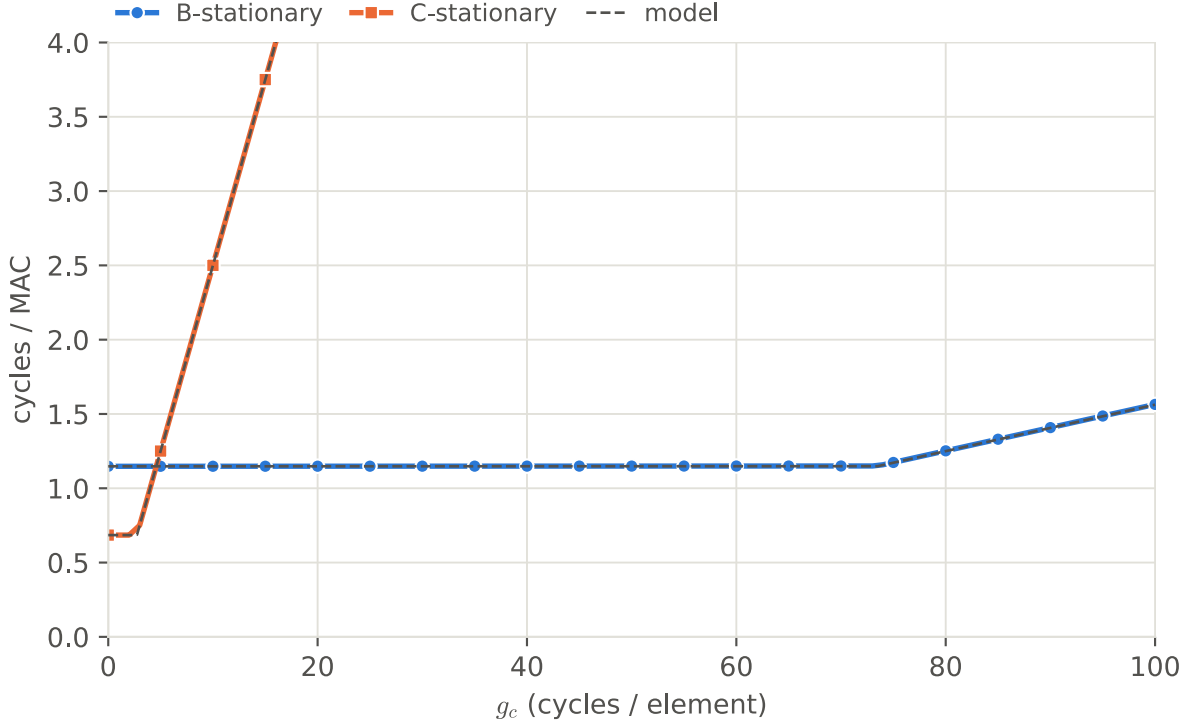


Figure 2: Runtime of the two dataflows against g_c on the tile 64×32 , measured (solid) and predicted by the model (dashed).

C -stationary is faster only when generation is almost free. At $g_c \leq 2$ it is $1.68 \times$ faster (0.685 against 1.148 cycles per MAC). From $g_c = 3$ on its runtime grows as $g_c/4$, and the two dataflows cross between $g_c = 4$ and $g_c = 5$. The model places the crossover where $g_c/R_M = \alpha_B$, at $g_c^* = R_M \cdot \alpha_B = 4.59$. Beyond it B -stationary is faster: $1.31 \times$ at $g_c = 6$, $6.5 \times$ at $g_c = 30$ and $16 \times$ from $g_c = 80$ on. The model matches both dataflows within 0.2% at every g_c .

B -stationary is not constant. Its generation cost $g_c/64$ is hidden behind α_B up to $g_c = T_M \cdot \alpha_B \approx 73$. Beyond it the runtime grows with slope $1/64$, 16 times more gently than C -stationary's $1/4$, because each generated element serves 64 rows instead of 4 .

Why C -stationary wins at $g_c = 0$. Both dataflows miss in L_1 equally often (0.0022 misses per MAC on this tile): they read the same tiles of A and C from memory. The difference is the L_1 hits. B -stationary loads and stores the register block of C at every step of K , 3/16 accesses per MAC. C -stationary keeps it in the registers for the whole of K and only loads A , 0.0645 accesses per MAC. At 4 cycles per L_1 access, 180 cycles per miss and 2 cycles per FIFO pop (Section 3.3 explains this counting in detail):

- $\alpha_B = 0.1875 \cdot 4 + 0.0022 \cdot 180 + 0.001 \cdot 2 = 1.148$,
- $\alpha_C = 0.0645 \cdot 4 + 0.0022 \cdot 180 + 0.0156 \cdot 2 = 0.685$,

where the last terms are the FIFO pops: C -stationary pops 16 times as often, since each popped block serves 4 rows instead of 64. Generation is the price of keeping C in the registers.

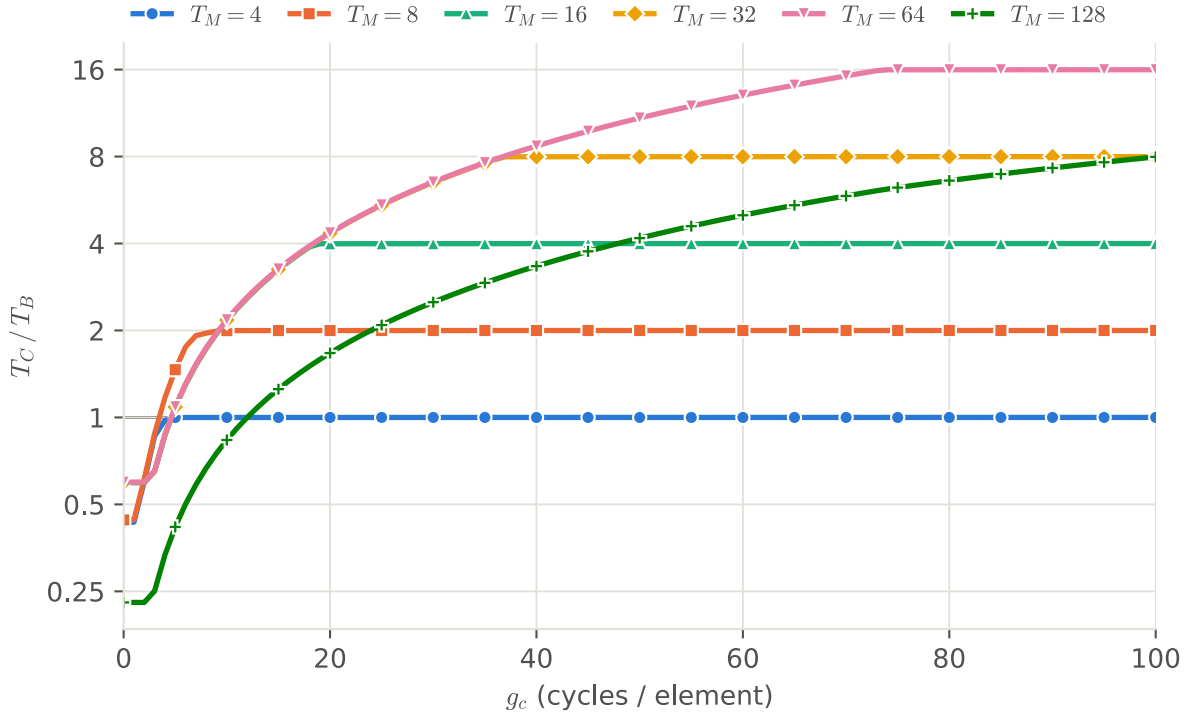


Figure 3: Runtime of C -stationary divided by the runtime of B -stationary against g_c , for $T_N = 32$ and six tile heights.

Other tiles. Figure 3 repeats the comparison for six tile heights ($T_M = 4, 8, 16, 32, 64, 128$) at $T_N = 32$, for every integer g_c from 0 to 100. Every line follows the same pattern:

1. At $g_c = 0$, C -stationary is faster: the ratio is 0.43–0.6, and 0.23 for $T_M = 128$, whose B -stationary tile no longer fits in L_1 ($\alpha_B = 2.99$).
2. The ratio crosses 1 at $g_c^* = R_M \cdot \alpha_B$, as for the 64×32 tile: at $g_c = 4$ for $T_M = 8$, at 5 for $T_M = 16$ to 64, and at 12 for $T_M = 128$.
3. Once both dataflows are limited by generation, the ratio settles at the ratio of their generation costs, T_M/R_M : exactly 2, 4, 8 and 16 for $T_M = 8, 16, 32, 64$. For $T_M = 128$, whose B -stationary tile is still limited by memory at $g_c = 100$, the ratio is 8 and still rising.
4. For $T_M = 4 = R_M$ the ratio is 1 from $g_c = 5$ on: B -stationary then reuses each generated element over only 4 rows, exactly like C -stationary, and has no advantage left. This confirms that the whole advantage of B -stationary is the reuse of B over T_M rows instead of R_M .

Each dataflow at its own best tile. Comparing the best tile of each dataflow (over $T_M \in \{4, 8, 16, \dots, 128\}$ in steps of 8, $T_N \in \{4, 8, 16, 32, 64\}$ and $g_c \in \{0, 1, \dots, 6, 8, 10, 15, 20, 30\}$) gives the same picture. C -stationary is faster only for $g_c \leq 3$ ($2.26 \times$ at $g_c \leq 1$, where both prefer the tile 8×64 and C -stationary costs 0.377 cycles per MAC). From $g_c = 4$ on, B -stationary is faster: $1.17 \times$ at $g_c = 4$, $2.57 \times$ at $g_c = 10$ and $7.7 \times$ at $g_c = 30$. The crossover again follows $R_M \cdot \alpha_B = 4 \cdot 0.854 = 3.4$, with α_B of B -stationary's best tile. Since C -stationary wins only when generating an element costs less than about 4 cycles, the rest of this report uses the B -stationary dataflow.

3.3 The Memory Cost α

With $g_c = 0$, generating B is free, and the runtime per MAC is the memory cost $\alpha(T_M, T_N)$ alone. We measured it for every T_M from 4 to 128 in steps of 4 and $T_N \in \{4, 8, 16, 32, 64\}$, with the default parameters (Table 2). These runs are also the calibration of the model: every later prediction uses the α measured here.

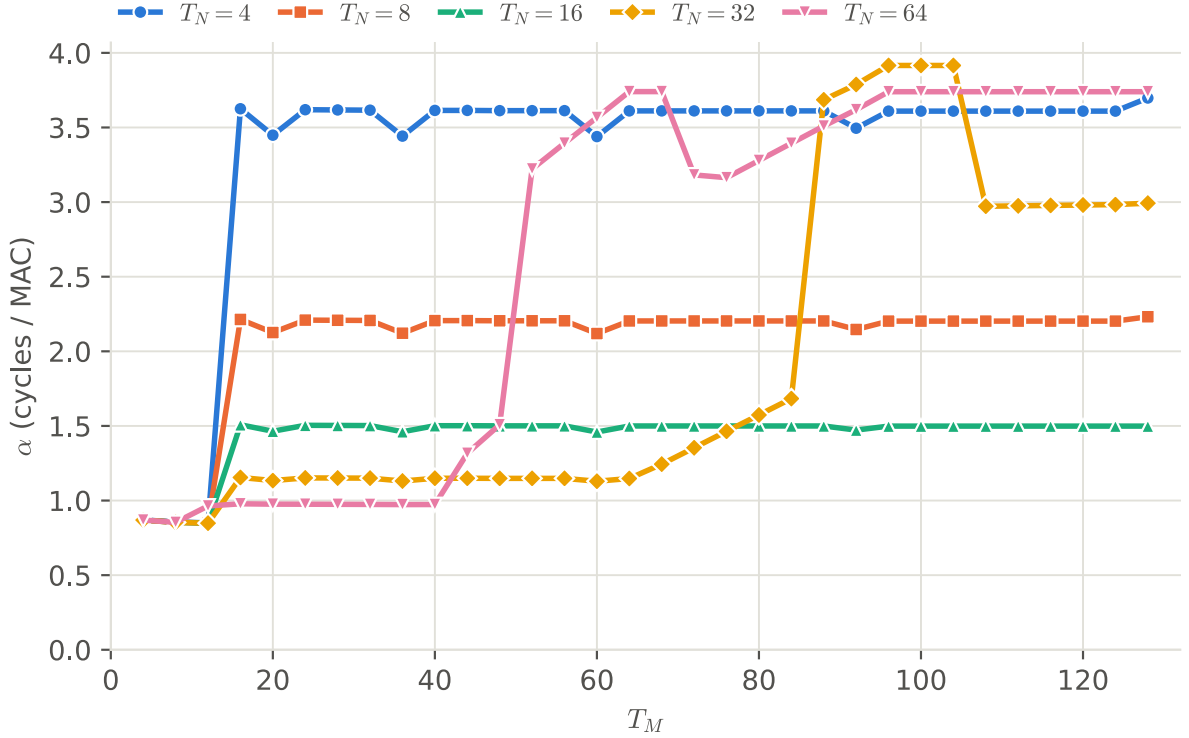


Figure 4: The memory cost α against the tile height T_M , one line per tile width T_N ($g_c = 0$, 16 KB fully associative L_1).

Figure 4 shows that α takes only a few levels, separated by sharp cliffs:

1. For $T_M \leq 12$, $\alpha \approx 0.85$ for every T_N .
2. From $T_M = 16$ on, α jumps to a plateau that depends only on T_N : 3.6, 2.2, 1.5, 1.15 and 0.97 for $T_N = 4, 8, 16, 32, 64$. Wider tiles are cheaper.
3. Wide tiles hit a second cliff, to $\alpha \approx 3.7$ – 3.9 : $T_N = 64$ at $T_M \approx 48$ – 52 , $T_N = 32$ at $T_M \approx 64$ – 88 . Narrower tiles ($T_N \leq 16$) do not reach it in this range.

4. The curves are not monotone: some tile heights are cheaper than both their neighbors (small dips at $T_M = 20, 36, 60, 92$, and larger drops for $T_N = 32$ at $T_M \geq 108$ and $T_N = 64$ at $T_M = 72, 76$).

Counting accesses. All of these levels follow from counting cache accesses in the weight-stationary loop nest. For every block of $R_M = 4$ steps of K and $R_N = 4$ columns, the core pops one register block of B and then sweeps the T_M/R_M register blocks of the tile, loading R_M lines of A and loading and storing R_M lines of C for each. That is $3R_M L_1$ accesses per R_M^3 MACs, or $3/16$ per MAC for any tile, which costs 0.75 cycles per MAC at 4 cycles per hit. The FIFO adds 2 cycles per pop, one pop per $R_M R_N$ elements of B shared by T_M rows, which is $1/(8T_M)$ cycles per MAC. Everything else is misses, at 180 cycles each, and a 64-byte line holds 16 elements:

- **A fits:** a tile of A is $T_M \times K$ elements, T_M KB at $K = 256$. Up to $T_M = 12$ it stays in L_1 while the core moves across the N/T_N tiles of its row, so A is read from memory only once: $1/(16N)$ misses per MAC. C is read once and written once: $1/(16K)$ misses per MAC.
- **A does not fit:** at $T_M = 16$ the tile of A alone fills the 16 KB of L_1 . It is then evicted before the next tile in its row starts and is read again from memory for every one of the N/T_N tiles: $1/(16T_N)$ misses per MAC, which costs $11.25/T_N$ cycles per MAC. This is the first cliff, and it is why the plateau depends only on T_N .
- **C does not fit:** each step of the inner loop sweeps the whole tile of C plus one line of A per row, about $T_M(\lceil T_N/16 \rceil + 1)$ lines. Once these exceed the 256 lines of L_1 , LRU evicts every line of C before it is used again, and the tile of C is reloaded from memory for every block of $R_M = 4$ steps of K : $1/(16R_M) = 1/64$ misses per MAC, or 2.81 cycles per MAC. With 5 lines per row ($T_N = 64$) the limit is $T_M \approx 51$, with 3 ($T_N = 32$) $T_M \approx 85$, and with 2 ($T_N \leq 16$) $T_M = 128$. This is the second cliff. It is complete at these limits and starts a little earlier, as the working set approaches the capacity of L_1 .

Table 3 compares these counts with the measurements. They agree within 0.3% in every regime: the memory cost is fully explained by how often each tile of A and C is read from memory.

T_N	A fits ($T_M = 8$)	A re-read ($T_M = 32$)	C reloaded ($T_M = 100$)
4	0.854 / 0.856	3.610 / 3.616	–
8	0.854 / 0.855	2.204 / 2.207	–
16	0.854 / 0.854	1.501 / 1.502	–
32	0.854 / 0.854	1.149 / 1.150	3.915 / 3.916
64	0.854 / 0.854	0.974 / 0.974	3.740 / 3.740

Table 3: α predicted by counting accesses / measured, in cycles per MAC. Predicted: $0.75 + 180 \cdot (A \text{ misses} + C \text{ misses}) + 1/(8T_M)$.

Tiles that do not divide the matrix. A tiling of $M = 192$ rows with $T_M = 72$ is two full tiles and an edge tile of 48 rows, and α is the row-weighted average of the two kinds of tile. At $T_N = 64$ the 48-row edge tile still fits in L_1 ($\alpha = 1.51$) while the full tiles do not (3.74), which gives $(144 \cdot 3.74 + 48 \cdot 1.51)/192 = 3.18$, exactly the measured value. The same mechanism explains every dip in Figure 4. $T_M = 20, 36, 60$ and 92 leave an edge tile of at most 12 rows, whose tile of A fits in L_1 . For $T_N = 32$, every T_M from 108 to 128 leaves an edge tile of 64 to 84 rows, whose tile of C still fits.

3.4 Memory Cost Against Generation Cost

Once generating B costs time, the model predicts the runtime as the larger of two costs: the memory cost α and the generation cost $g_c \cdot \lceil M/T_M \rceil / M$. Figure 5 shows both for $T_N = 32$ and T_M from 4 to 128 in steps of 4, together with the runtime measured at $g_c = 30$ and $g_c = 200$, with the default configuration otherwise (Table 2).

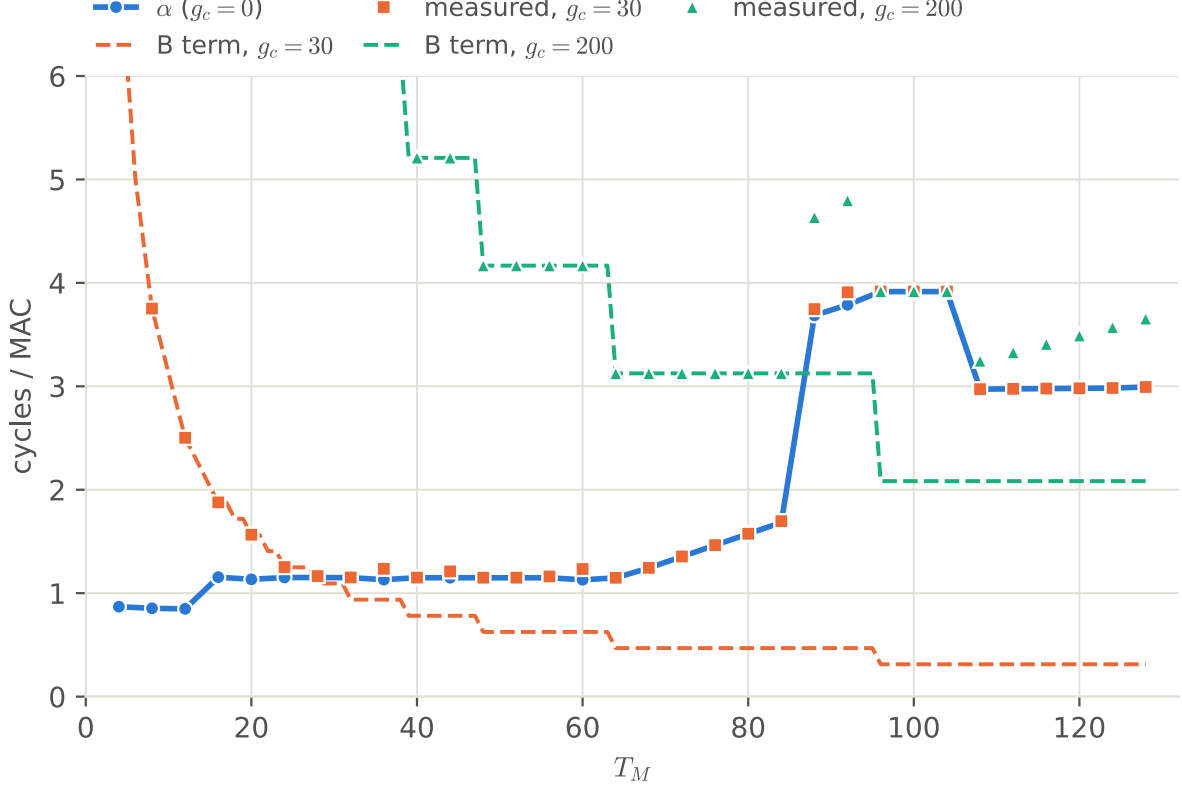


Figure 5: The two costs of the model against T_M ($T_N = 32$): the memory cost α measured at $g_c = 0$ (solid), the generation cost $g_c \cdot \lceil M/T_M \rceil / M$ (dashed), and the measured runtime (markers) at $g_c = 30$ and $g_c = 200$.

The generation cost is a staircase. B is generated once per row of tiles, so its cost changes only when the number of rows of tiles changes. With $M = 192$, every T_M from 64 to 95 needs three rows of tiles and costs $g_c/64$, and every T_M from 96 to 191 needs two and costs $g_c/96$. A taller tile within a step does not reduce the generation cost, it only makes the last row of tiles shorter. The curve g_c/T_M touches the staircase where T_M divides M and lies below it elsewhere.

$g_c = 30$. For $T_M \leq 24$ the generation cost is above α , and the measured runtime follows the staircase. From $T_M = 28$ on, α is the larger cost, and the runtime follows α , including its cliffs. The fastest tile is $T_M = 64$ (1.149 cycles per MAC), but every height on the plateau of α that divides M ($T_M = 32, 48, 64$) is within 0.3% of it. Generation is then completely hidden behind the memory accesses.

$g_c = 200$. The generation cost is above α up to $T_M = 84$, and the runtime follows the staircase. Every T_M from 64 to 84 costs the same 3.127 cycles per MAC, the generation cost of three rows of tiles: these are the fastest tiles. Two rows of tiles ($T_M \geq 96$) would cut the generation cost to $g_c/96 = 2.08$, but those tiles are past the second cliff of α (at least partly), and they cost at least 3.25 cycles per MAC.

How exact. For every T_M that divides M , the measurements agree with the model, $\max(\alpha, g_c \cdot \lceil M/T_M \rceil / M)$ plus the startup term, within 0.2%. The other tile heights are sometimes slower than the model predicts: by 5–9% at $T_M = 36, 44, 60$ for $g_c = 30$, and by up to 26% at $T_M = 88, 92$ and 108–128 for $g_c = 200$. All of them leave a short last row of tiles, and Section 3.5 explains the difference.

The figure shows the trade-off behind every tile choice in this report. Taller tiles reduce the generation cost, because each element of B is reused by more rows, but they increase the memory cost once the tiles no longer fit in L_1 . The fastest tile is the lowest point of the larger of the two costs, and it moves to taller tiles as g_c grows.

3.5 Refining The Model

3.6 Choosing The Tile

To find the best tile at each generation cost, we simulated every tile with T_M from 4 to 192 and T_N from 4 to 128, both in steps of 4 (1536 tiles), at 42 values of g_c from 0 to 600 (every integer up to 10, then increasingly coarser steps), with the default configuration otherwise (Table 2): 64,512 runs in all. Figure 6 shows the measured best tile (T_M^*, T_N^*) and the tile the model picks, using α from the $g_c = 0$ runs.

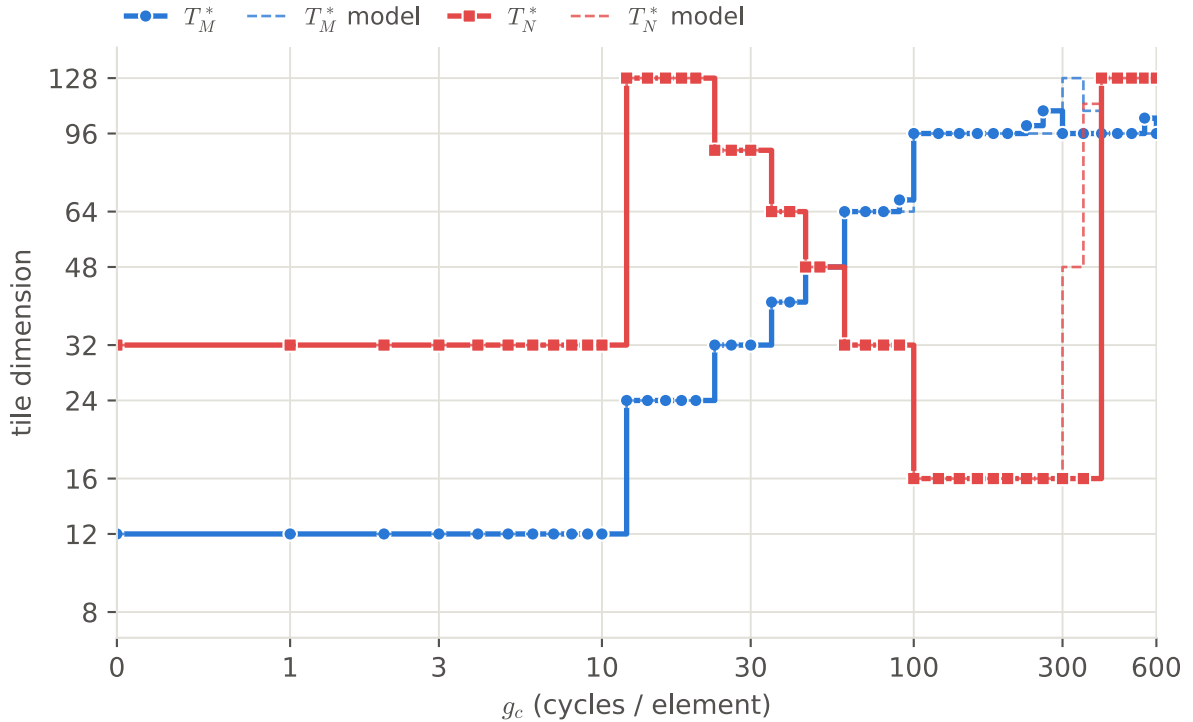


Figure 6: The best tile against g_c : its height T_M^* and width T_N^* , measured (solid) and picked by the model (dashed).

The best tile changes shape. As g_c grows, the best tile gets taller and narrower:

g_c	0–10	12–20	23–30	35–40	45–50	60–80
best tile	12×32	24×128	32×88	40×64	48×48	64×32

and from $g_c = 100$ on it is 96×16 or a tile that ties with it. Both trends follow from the two costs of Section 3.4:

- **T_M grows** so that generation stays hidden: the generation cost $g_c \cdot \lceil M/T_M \rceil / M$ must stay below α , which is about 1 cycle per MAC for good tiles, so the tile needs more rows as g_c grows. At $g_c = 12$, the tile 12×32 would cost 1 cycle per MAC for generation alone, more than its $\alpha = 0.85$.
- **T_N shrinks** so that the tile still fits in L_1 . Once the tile of A no longer fits ($T_M \geq 16$), a wider tile reads A fewer times (Section 3.3) and is cheaper, up to the second cliff, where the tile of C and a line of A per row no longer fit. A taller tile reaches that cliff at a smaller width. Every best tile from 24×128 to 96×16 has the lowest α of all widths at its height, and no wider tile of the same height is cheaper: its tile of C plus one line of A per row fill roughly 190 to 225 of the 256 lines of L_1 .

Ties. Where the best tile is not sharply defined, the exact winner is a matter of noise:

- Up to $g_c = 6$, the tile of A fits in L_1 , the width hardly matters, and 34 to 38 tiles (all with $T_M = 12$) are within 1% of the best.
- From $g_c = 100$ on, every tile with two rows of tiles (T_M from 96 to 128) has the same generation cost, and the best height alternates between them.
- From $g_c = 400$ on, generation dominates every good tile, so the width no longer matters either. Up to 263 tiles are within 1% of the best, and the jump of T_N^* to 128 is one of these ties.

The height is capped by L_1 . A single row of tiles ($T_M = M = 192$) would generate B only once, but its tile of C plus a line of A per row needs at least 384 lines, more than L_1 holds. Every access then misses ($\alpha = 23.25$), and this tile becomes the fastest only when generation is even more expensive: between $g_c = 2000$ and 2500 in a coarser sweep (steps of 8 in T_M).

The model. The tile picked by the model is the best tile, or ties with it within 0.1%, at 39 of the 41 values of $g_c > 0$. The two exceptions are $g_c = 300$ and 350, where the picked tiles (128×48 and 108×112) are 29% and 7% slower than the best. Both leave a short last row of tiles, the case Section 3.5 explains.

How much does the choice matter? The default tile 16×32 is 36% slower than the best tile for $g_c \leq 10$, twice as slow at $g_c = 30$, four times as slow at $g_c = 80$, and six times as slow from $g_c \approx 160$ on.

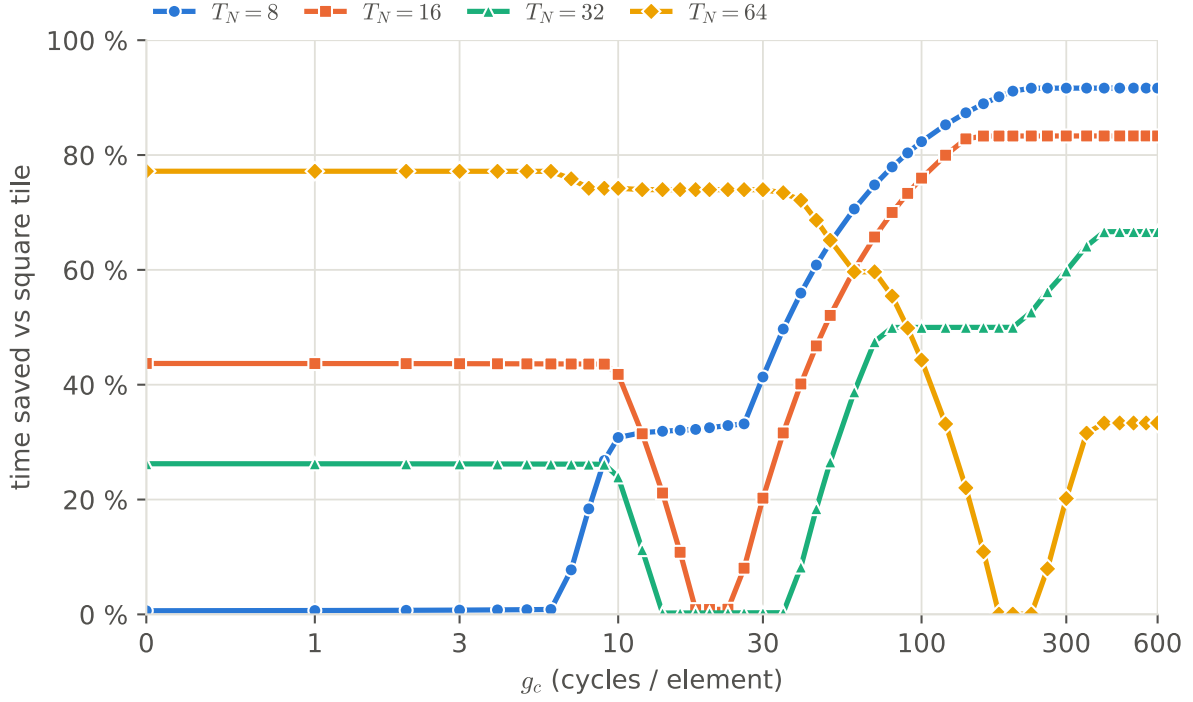


Figure 7: Time saved by the best tile height over the square tile $T_M = T_N$, for four fixed tile widths T_N .

Against square tiles. Square tiles are optimal when all matrices cost the same (Section 1.3). Figure 7 fixes the width T_N , as a hardware vector width or the matrix shape might, and compares the best height T_M with the square tile $T_M = T_N$. The gain is the time saved, $(T_{\text{square}} - T_{\text{best}})/T_{\text{square}}$:

- **Small g_c :** the gain is how badly the square tile fits in L_1 . A 64×64 tile is past the second cliff ($\alpha = 3.74$) while 8×64 costs 0.85, a gain of 77%. A 8×8 tile already fits, and the gain is about 1%.
- **Large g_c :** the gain tends to a fixed limit. The square tile needs $\lceil 192/T_N \rceil$ rows of tiles, the best tile only two, so the gain tends to $1 - 2/\lceil 192/T_N \rceil$: 92%, 83%, 67% and 33% for $T_N = 8, 16, 32, 64$. These are exactly the measured values from $g_c = 400$ on.
- **In between,** the gain drops to zero where the square tile happens to cost as much as the best height, for example 32×32 for g_c from 14 to 35, where it sits on the same plateau of α as the best height, 64.

Comparing the overall best tile with the best square tile of any size gives smaller but still large gains. Square tiles are as good for $g_c \leq 10$ (12×12), at $g_c = 45\text{--}50$ (where 48×48 is the best tile) and from $g_c = 400$ on (where generation dominates). In between, the best tile is up to 50% faster (96×16 against 48×48 at $g_c = 160$), and 25–50% faster for g_c from 80 to 200. A square tile cannot be tall, to generate B few times, and narrow, to fit in L_1 , at the same time.

3.7 Set-Associative Caches

4 Discussion

5 Conclusion

Bibliography

- [1] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2017.
- [2] J.-W. Hong and H. T. Kung, “I/O complexity: The red-blue pebble game,” in *Proceedings of the 13th Annual ACM Symposium on Theory of Computing (STOC)*, 1981, pp. 326–333.
- [3] Y.-H. Chen, J. Emer, and V. Sze, “Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks,” in *Proceedings of the 43rd International Symposium on Computer Architecture (ISCA)*, 2016, pp. 367–379.
- [4] N. Halko, P.-G. Martinsson, and J. A. Tropp, “Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions,” *SIAM Review*, vol. 53, no. 2, pp. 217–288, 2011.
- [5] D. P. Woodruff, “Sketching as a Tool for Numerical Linear Algebra,” *Foundations and Trends in Theoretical Computer Science*, vol. 10, no. 1–2, pp. 1–157, 2014.
- [6] P.-G. Martinsson and J. A. Tropp, “Randomized numerical linear algebra: Foundations and algorithms,” *Acta Numerica*, vol. 29, pp. 403–572, 2020.
- [7] R. Murray et al., “Randomized Numerical Linear Algebra: A Perspective on the Field With an Eye to Software,” *arXiv preprint arXiv:2302.11474*, 2023, [Online]. Available: <https://arxiv.org/abs/2302.11474>
- [8] W. B. Johnson and J. Lindenstrauss, “Extensions of Lipschitz mappings into a Hilbert space,” in *Contemporary Mathematics*, vol. 26, 1984, pp. 189–206.
- [9] A. Rahimi and B. Recht, “Random Features for Large-Scale Kernel Machines,” in *Advances in Neural Information Processing Systems 20 (NIPS 2007)*, 2007.
- [10] D. Achlioptas, “Database-friendly random projections: Johnson-Lindenstrauss with binary coins,” *Journal of Computer and System Sciences*, vol. 66, no. 4, pp. 671–687, 2003.
- [11] “Dense Times Random / Mixed Precision Matrix Multiplication Tiling Schemes (unpublished notes).” 2021.
- [12] “Multiplication of Matrices with Asymmetric Access Costs (unpublished slides).” 2022.

6 Appendix

6.1 Implementation & Build Environment

The full simulator source code, build configuration, and experiment reproduction scripts are available on GitHub:

https://github.com/gevorgyg/asymm_tiling

The simulator is built with CMake (version ≥ 3.16) targeting the C++20 language standard. The implementation avoids heavy runtime frameworks and utilizes header-only libraries:

- **CLI11:** Provides command-line option parsing with automatic type conversion and validation.
- **toml++:** Parses TOML configuration files (`config.toml`), allowing experiment parameters to be specified declaratively without recompilation.
- **fmt & spdlog:** High-performance structured logging library used to log cycle events and format aligned metrics tables.